## 1. Write a program to generate public and private key using OpenSSL.

**Steps to execute**

1. **Install OpenSSL**
    - Download a Windows OpenSSL installer (for example “Win64 OpenSSL” from a trusted provider such as Shining Light Productions / SLProWeb).
    - Install it (note the install folder, e.g. C:\Program Files\OpenSSL-Win64).
    - Add the OpenSSL bin folder to your PATH:
        - Start → *Edit system environment variables* → *Environment Variables* → edit **Path** → New → paste C:\Program Files\OpenSSL-Win64\bin → OK.
    - Re-open PowerShell after editing PATH.
2. **Verify OpenSSL**
    - Open PowerShell and run:
    - openssl version
    - You should see something like OpenSSL 3.0.x .... If you get an error, PATH wasn’t set correctly.
3. **Create a folder and save script**
    - In PowerShell:
    - mkdir C:\keys-short
    - cd C:\keys-short
    - Create a file named generate_keys_short.ps1 and paste the **Whole code** (below) into it.
4. **Run the script**
    - In PowerShell (normal window):
    - cd C:\keys-short
    - Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
    - .\generate_keys_short.ps1
    - The script will generate two files in C:\keys-short:
        - private_key.pem (private key)
        - public_key.pem (public key)
5. **Inspect keys**
    - View private key (careful — do not share):
    - type .\private_key.pem | more
    - View public key:
    - type .\public_key.pem | more

---

**Explanation**

openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
Generates a 2048-bit RSA **private key** and writes it to private_key.pem. genpkey is the modern OpenSSL command for key generation.

- openssl rsa -in private_key.pem -pubout -out public_key.pem
  Reads the private key and extracts the corresponding **public key** in PEM format (public_key.pem).
- PowerShell Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
  Temporarily allows running the script in this PowerShell session if script execution is restricted.
- Files:
  - private_key.pem — **Keep secret**. Used to sign/decrypt.
  - public_key.pem — Shareable. Used to verify/encrypt.

**Security notes**

- Don't upload or share private_key.pem. Store it securely (NTFS permissions, BitLocker, etc.).
- For stronger security in production, use 3072/4096 bits or prefer elliptic-curve keys (e.g., openssl ecparam -name prime256v1 -genkey -noout -out key.pem).
- If you want the private key encrypted with a passphrase, you can convert it with openssl pkcs8 -topk8 -in private_key.pem -v2 aes-256-cbc -out private_key_encrypted.pem -passout pass:YOURPASS (but avoid passing passphrases on the command line in scripts).

---

**Whole code (very short PowerShell script)**

Save exactly this as generate_keys_short.ps1 in C:\keys-short and run as shown above.

```
# generate_keys_short.ps1

# Very short script to create an RSA private key and its public key using OpenSSL (Windows)


# Generate 2048-bit RSA private key

openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048


# Extract public key (PEM)

openssl rsa -in private_key.pem -pubout -out public_key.pem


Write-Host "Done. Files created:"

Write-Host " - private_key.pem  (KEEP SECRET)"

Write-Host " - public_key.pem   (shareable)"
```

## 2. Write a program to create a simple Blockchain using Python

**Steps to Execute**

1. **Install Python**
    - Download from https://python.org
    - During installation, check **"Add Python to PATH"**.
    - Verify:
    - python --version
2. **Create a folder**
3. C:\simple-blockchain
4. **Create the Python file**
    - Inside that folder, make a file named:
    - blockchain_short.py
    - Paste the code from the "Whole Code" section below.
5. **Run the program**
   Open CMD or PowerShell:
6. cd C:\simple-blockchain
7. python blockchain_short.py

You will see:

- Genesis block
- A new block being mined
- Chain printed
- Validity check

---

**Explanation (simple and separate)**

**✓ What is a Blockchain?**

A blockchain is just a **list of blocks**, where each block has:

- Some data
- A timestamp
- A hash
- The previous block's hash

Because each block stores the previous hash, the chain becomes tamper-evident.

**✓ What this tiny program does**

This very small Python program:

1. Creates a **Block** class
2. Creates a simple **Blockchain** list
3. Adds a **genesis block** (first block)
4. Creates another block with custom data
5. Links blocks using **previous_hash**
6. Prints the full chain

**✓ Hashing (why it's important)**

Each block's data is hashed using SHA-256.
If someone modifies a block, its hash changes → chain becomes invalid.

**✓ Why this program is short**

You asked for a *very short* example.
So:

- No mining difficulty
- No proof of work
- No transactions
- No wallets
- Only basic hashing and block linking

This is the **minimal educational blockchain**.

---

**Whole Code (very short version)**

Save as **blockchain_short.py**:

```
import hashlib, time


class Block:
    def __init__(self, data, previous_hash="0"):
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        block_string = f"{self.timestamp}{self.data}{self.previous_hash}"
```

```
        return hashlib.sha256(block_string.encode()).hexdigest()

# Simple blockchain
chain = []

# Genesis block
chain.append(Block("Genesis Block"))

# Add a new block
chain.append(Block("Second Block", chain[-1].hash))

# Print the blockchain
for i, block in enumerate(chain):
    print(f"\nBlock {i}")
    print("Timestamp:", block.timestamp)
    print("Data:", block.data)
    print("Previous Hash:", block.previous_hash)
    print("Hash:", block.hash)
```

## 3. Develop and test smart contract on local Blockchain.

**Steps to execute (Windows — very short, step-by-step)**

Follow these exact commands in **PowerShell** (Windows). Each line is a separate command.

1. Install Node.js (if not installed)
    - Download & install Node.js 16+ from https://nodejs.org and check **Add to PATH** during installation.
    - Verify:
    - node --version
    - npm --version
2. Create project folder and initialize
3. mkdir C:\local-solidity
4. cd C:\local-solidity
5. npm init -y
6. Install Hardhat and helpers (dev deps)
7. npm install --save-dev hardhat @nomiclabs/hardhat-ethers ethers chai
8. Create a minimal Hardhat project
9. npx hardhat
    - When prompted choose **"Create a basic sample project"** (use default answers). This creates folders contracts/, test/, and a hardhat.config.js. If it asks to install sample dependencies, accept (we already installed main ones).
10. Replace files with the *Very Short* contract & test (see **Whole code** section). Create/overwrite:
    - contracts/SimpleStorage.sol
    - test/simpleStorage-test.js
    - hardhat.config.js (if your npx hardhat created a different config, replace with the one in Whole code or merge values)
11. Run tests (this starts Hardhat's local in-memory blockchain and runs the test)
12. npx hardhat test
13. Optional — run a local node (persistent) for manual deployment
14. npx hardhat node

In another PowerShell window you can run scripts that deploy to http://127.0.0.1:8545.

---

**Explanation**

What you are doing

- You'll use **Hardhat** (a lightweight Ethereum dev environment) to develop and test a Solidity smart contract locally.
- Hardhat provides an in-memory local blockchain when you run npx hardhat test (no external network required).

Key pieces

- **SimpleStorage.sol** — tiny Solidity contract with one uint state variable and functions set and get.
- **Test (JavaScript)** — uses ethers from Hardhat to:
    - compile the contract,
    - deploy it to the local Hardhat network,
    - call set(42),
    - read get() and assert the stored value equals 42.

Why this works locally

- npx hardhat test compiles contracts, starts a local ephemeral node, deploys the contract, runs the test, and then tears down — perfect for fast dev iterations.
- No external blockchain or wallet configuration needed.

Notes & tips

- Tests run fast and are deterministic on Hardhat.
- For manual interactive tests or using MetaMask, run npx hardhat node and deploy to http://127.0.0.1:8545 from a deployment script and import one of the node accounts into MetaMask.
- For production-like testing, add more tests: access control, reverts, edge cases.

Security reminder

- This is only a development setup. Do **not** use the same private keys, RPC endpoints, or contract addresses on mainnet without proper auditing and security checks.

---

**Whole code (put these files in the project)**

Below are the minimal files. Create them under C:\local-solidity as shown.

**contracts/SimpleStorage.sol**

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;


/// @title Very small storage contract (demo)

contract SimpleStorage {

    uint256 private value;
```

```
    function set(uint256 _v) external {
        value = _v;
    }

    function get() external view returns (uint256) {
        return value;
    }
}
```

---

**test/simpleStorage-test.js**

```
const { expect } = require("chai");
const { ethers } = require("hardhat");

describe("SimpleStorage (very short)", function () {
  it("stores and retrieves a value", async function () {
    const SimpleStorage = await ethers.getContractFactory("SimpleStorage");
    const instance = await SimpleStorage.deploy();
    await instance.deployed();

    // set value to 42
    const tx = await instance.set(42);
    await tx.wait();

    // read stored value
    const stored = await instance.get();
    expect(stored.toNumber()).to.equal(42);
  });
});
```

---

**hardhat.config.js**

```
require("@nomiclabs/hardhat-ethers");
```

```
module.exports = {
  solidity: "0.8.20", // modern stable compiler; adjust if needed
  paths: {
    sources: "./contracts",
    tests: "./test",
  },
  networks: {
    hardhat: {
      // default in-memory network for tests
    }
  }
};
```

---

**(Optional) package.json — only if you want a quick npm test script (you already ran npm init -y)**

If you want npm test to run Hardhat tests, add this script to package.json:

```
{
  "scripts": {
    "test": "npx hardhat test"
  }
}
```

Then run:

```
npm test
```

## 4. Develop and test smart contract on Ethereum test networks.

**Steps to execute**

Follow these in **PowerShell** (each line is a separate command unless grouped):

1. Install Node.js (16+) and npm
    - Download from https://nodejs.org and install (choose “Add to PATH”).
    - Verify:
2. node --version
3. npm --version
4. Create project folder and initialize:

mkdir C:\sepolia-deploy

cd C:\sepolia-deploy

npm init -y

3. Install Hardhat + ethers:

npm install --save-dev hardhat @nomiclabs/hardhat-ethers ethers dotenv

4. Create a minimal Hardhat project (creates sample files):

npx hardhat

- Choose **Create an empty hardhat.config.js** or basic project. Hardhat docs show config options and network setup.

5. Create an Alchemy (or Infura/QuickNode) free account and get a **Sepolia RPC URL / API key** (you’ll use it in step 8). Alchemy provides Sepolia endpoints and a Sepolia faucet to obtain test ETH.
6. Install MetaMask (browser extension) and create/import an account. Switch to Sepolia network in MetaMask (enabled by default or add via RPC).
7. Get Sepolia ETH from a faucet (Alchemy/other) and send to the MetaMask address to pay gas.
8. Create a .env file in the project root with:

SEPOLIA_RPC_URL="https://eth-sepolia.alchemyapi.io/v2/yourKeyHere"

PRIVATE_KEY="0x...."  # one of your MetaMask account private keys (for testnet only)

**Security note:** keep .env local and don’t commit it to git.

9. Create the contract and deploy script (see **Whole code** section for the 3 tiny files: contracts/SimpleStorage.sol, scripts/deploy.js, hardhat.config.js).
10. Compile and deploy to Sepolia:

npx hardhat compile

npx hardhat run scripts/deploy.js --network sepolia

- Hardhat will send the deployment transaction via your provider and private key.

11. Verify deployment by checking the contract address on Sepolia block explorer (e.g., Etherscan Sepolia) or by calling the contract from a small script or etherscan UI.

---

**Explanation**

What we're doing (overview)

- Writing a **tiny Solidity contract** (SimpleStorage: set/get a uint).
- Using **Hardhat** as the development tool to compile, run, and deploy the contract.
- Deploying to **Sepolia testnet** via an RPC provider (Alchemy/Infura/QuickNode).
- Funding the deploying account with **Sepolia ETH** from a faucet so transactions can pay gas.

Why Sepolia?

- Sepolia is the recommended public testnet for dapp development (post-Merge) and is broadly supported by node providers and MetaMask. Use Sepolia instead of older/deprecated testnets. Metana+1

Key pieces

- **hardhat.config.js** — tells Hardhat which compiler and networks to use.
- **.env** — stores Sepolia RPC URL and the deployer private key (test-only).
- **scripts/deploy.js** — simple ethers-based deployment script invoked by Hardhat.
- **contracts/SimpleStorage.sol** — tiny contract to demonstrate deployment.

Security & practical notes

- Never use a real/mainnet private key or mainnet funds in .env for tests — use test accounts only.
- Keep your .env out of version control (.gitignore it).
- For repeated testing, use npx hardhat node (local node) or the Sepolia testnet for cross-device testing and MetaMask integration. Hardhat docs explain deploying to public networks. Hardhat

---

**Whole code**

Create these files under C:\sepolia-deploy.

---

**1) contracts/SimpleStorage.sol**

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;


contract SimpleStorage {

    uint256 private value;
```

```
    function set(uint256 _v) external {
        value = _v;
    }

    function get() external view returns (uint256) {
        return value;
    }
}
```

---

**2) scripts/deploy.js**

```
// scripts/deploy.js
require('dotenv').config();
const hre = require("hardhat");

async function main() {
  const SimpleStorage = await hre.ethers.getContractFactory("SimpleStorage");
  console.log("Deploying SimpleStorage...");
  const instance = await SimpleStorage.deploy();
  await instance.deployed();
  console.log("SimpleStorage deployed to:", instance.address);

  // optional quick test: set then get
  const tx = await instance.set(42);
  await tx.wait();
  const val = await instance.get();
  console.log("Value after set:", val.toString());
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

---

**3) hardhat.config.js**

```
require("@nomiclabs/hardhat-ethers");
require('dotenv').config();

const { SEPOLIA_RPC_URL, PRIVATE_KEY } = process.env;

module.exports = {
  solidity: "0.8.20",
  networks: {
    sepolia: {
      url: SEPOLIA_RPC_URL || "",
      accounts: PRIVATE_KEY ? [PRIVATE_KEY] : []
    }
  }
};
```

---

**4) .env (example — DO NOT share or commit)**

```
SEPOLIA_RPC_URL="https://eth-sepolia.alchemyapi.io/v2/yourKeyHere"
PRIVATE_KEY="0xyourprivatekeyhere"
```

---

## 5. Design and develop Cryptocurrency for multiple user using python.

**Steps to execute**

1. **Install Python**
    - Download and install Python 3.8+ from https://python.org.
    - During install check **"Add Python to PATH"**.
    - Verify in PowerShell or Command Prompt:
    - python --version
2. **Create a working folder**
    - Open PowerShell:
    - mkdir C:\crypto-multi
    - cd C:\crypto-multi
3. **Save the script**
    - Create a file named crypto_multi.py in C:\crypto-multi.
    - Copy the code from the **Whole code** section below and paste it into crypto_multi.py. Save the file.
4. **Run the program**
    - In PowerShell:
    - cd C:\crypto-multi
    - python crypto_multi.py
    - The script runs a short demo: creates users, funds the system via mining rewards, makes transfers, prints balances and the chain, and checks validity.
5. **Experiment**
    - Edit crypto_multi.py to change difficulty, mining_reward, add more users, or simulate more transactions. Re-run to observe behavior.

---

**Explanation**

**Overview**

This tiny prototype implements:

- A minimal **Block** with index, timestamp, transactions, previous_hash, nonce, and hash.
- A simple **Blockchain** with Proof-of-Work (PoW) mining (leading zeroes).
- A **Cryptocurrency** manager that tracks multiple users (addresses), creates transactions, and mines pending transactions awarding a miner reward.

**Components**

1. **User addresses**

- Created with create_user(name). Internally uses a random 32-byte secret + SHA-256 to produce a unique address (hex). No public/private key cryptography or signatures are used in this short demo (signature is a TODO field).

2. **Transactions**
   - Minimal dicts: {sender, recipient, amount}.
   - Sender "SYSTEM" is used for mining reward minting.
3. **Mining / Proof-of-Work**
   - difficulty controls how many leading 0 characters are required in a block hash.
   - mine_pending_transactions(miner_address) creates a block with pending transactions, finds a valid nonce, appends the block, and places the mining reward as the single pending transaction for the next block.
4. **Balances**
   - get_balance(address) scans the blockchain and pending transactions to compute balance.
5. **Validation**
   - is_chain_valid() checks block hashes and previous_hash chain links, and enforces the difficulty for each block.

**Why this is educational (not production)**

- No cryptographic signing of transactions (so anyone could claim any sender).
- No P2P network, mempool, or consensus beyond simple single-node PoW.
- Mining difficulty and reward are configurable but simplistic.
- Good next steps: add ECDSA signatures, persistent storage, networking, and a proper mempool & validation rules.

---

**Whole code (save as crypto_multi.py)**

```
# crypto_multi.py

# Very short multi-user cryptocurrency prototype (Windows-friendly)

# Run: python crypto_multi.py


import hashlib, json, time, secrets


# --- Block ---

class Block:

    def __init__(self, index, transactions, previous_hash=''):

        self.index = index
```

```
        self.timestamp = time.time()
        self.transactions = transactions  # list of dicts
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        block_string = json.dumps({
            'index': self.index,
            'timestamp': self.timestamp,
            'transactions': self.transactions,
            'previous_hash': self.previous_hash,
            'nonce': self.nonce
        }, sort_keys=True).encode()
        return hashlib.sha256(block_string).hexdigest()

    def mine(self, difficulty):
        target = '0' * difficulty
        while not self.hash.startswith(target):
            self.nonce += 1
            self.hash = self.calculate_hash()
        print(f"  Mined block {self.index} with nonce={self.nonce}, hash={self.hash[:16]}...")

# --- Blockchain ---
class Blockchain:
    def __init__(self, difficulty=2, mining_reward=50):
        self.chain = [self._create_genesis()]
        self.difficulty = difficulty
        self.pending_transactions = []
        self.mining_reward = mining_reward

    def _create_genesis(self):
```

```
        genesis = Block(0, [{"message": "Genesis Block"}], "0")
        genesis.hash = genesis.calculate_hash()
        return genesis

    def get_latest(self):
        return self.chain[-1]

    def add_transaction(self, tx):
        if 'sender' not in tx or 'recipient' not in tx or 'amount' not in tx:
            raise ValueError("Transaction must have sender, recipient, amount")
        self.pending_transactions.append(tx)

    def mine_pending_transactions(self, miner_address):
        if not self.pending_transactions:
            print("  No transactions to mine.")
            return
        new_block = Block(len(self.chain), self.pending_transactions.copy(), self.get_latest().hash)
        print(f"Mining block {new_block.index} with {len(new_block.transactions)} tx(s)...")
        new_block.mine(self.difficulty)
        self.chain.append(new_block)
        # reward to miner in next round
        self.pending_transactions = [{'sender': 'SYSTEM', 'recipient': miner_address, 'amount':
self.mining_reward}]

    def get_balance(self, address):
        balance = 0
        for block in self.chain:
            for tx in block.transactions:
                if tx.get('sender') == address:
                    balance -= tx.get('amount', 0)
                if tx.get('recipient') == address:
                    balance += tx.get('amount', 0)
        # include pending txs as unconfirmed balance
```

```
        for tx in self.pending_transactions:
            if tx.get('sender') == address:
                balance -= tx.get('amount', 0)
            if tx.get('recipient') == address:
                balance += tx.get('amount', 0)
        return balance

    def is_chain_valid(self):
        for i in range(1, len(self.chain)):
            cur = self.chain[i]
            prev = self.chain[i-1]
            if cur.hash != cur.calculate_hash():
                print(f"Invalid hash at block {i}")
                return False
            if cur.previous_hash != prev.hash:
                print(f"Invalid previous_hash at block {i}")
                return False
            if not cur.hash.startswith('0'*self.difficulty):
                print(f"Block {i} fails difficulty check")
                return False
        return True

# --- Simple multi-user cryptocurrency manager ---
class SimpleCrypto:
    def __init__(self, difficulty=2, reward=50):
        self.bc = Blockchain(difficulty=difficulty, mining_reward=reward)
        self.users = {}  # name -> address

    def create_user(self, name):
        # deterministic-ish address: sha256(random + name)
        rand = secrets.token_hex(16)
        addr = hashlib.sha256((rand + name).encode()).hexdigest()
```

```
        self.users[name] = addr
        print(f"Created user '{name}' -> {addr[:12]}...")
        return addr

    def send(self, sender_addr, recipient_addr, amount):
        if amount <= 0:
            raise ValueError("Amount must be positive")
        tx = {'sender': sender_addr, 'recipient': recipient_addr, 'amount': amount}
        self.bc.add_transaction(tx)
        print(f"  Queued tx: {sender_addr[:8]}... -> {recipient_addr[:8]}... : {amount}")

    def mine(self, miner_name):
        miner_addr = self.users.get(miner_name)
        if not miner_addr:
            raise ValueError("Miner must be a created user")
        self.bc.mine_pending_transactions(miner_addr)

    def balance(self, name_or_addr):
        addr = self.users.get(name_or_addr, name_or_addr)
        return self.bc.get_balance(addr)

# --- Demo usage ---
def demo():
    print("Simple multi-user cryptocurrency demo\n")
    sc = SimpleCrypto(difficulty=3, reward=25)  # adjust difficulty as needed

    # Create users
    alice = sc.create_user("Alice")
    bob = sc.create_user("Bob")
    miner = sc.create_user("Miner1")

    # Alice receives initial coins via system (simulate faucet by directly adding a SYSTEM tx)
```

```
    sc.bc.add_transaction({'sender': 'SYSTEM', 'recipient': alice, 'amount': 100})
    # Mine to include faucet tx
    sc.mine("Miner1")

    # Alice sends Bob some coins
    sc.send(alice, bob, 20)
    sc.send(alice, bob, 5)

    # Miner mines pending transactions and earns reward
    sc.mine("Miner1")

    # Print balances
    for name, addr in sc.users.items():
        print(f"Balance {name}: {sc.balance(addr)}")

    # Print chain summary
    print("\nChain summary:")
    for b in sc.bc.chain:
        print(f" Block {b.index} | txs={len(b.transactions)} | hash={b.hash[:12]}...")

    print("\nChain valid?:", sc.bc.is_chain_valid())

if __name__ == "__main__":
    demo()
```

## 6. Write and deploy chain code in Hyperledger Fabric.

**Steps to execute**

Follow these commands **inside WSL2 (Ubuntu)**. Each line is a separate command unless grouped.

1. **Install prerequisites on Windows**
    - Enable/install **WSL2** and install **Ubuntu** from Microsoft Store. (Search “WSL2 install” Microsoft docs.)
    - Install **Docker Desktop for Windows** and enable WSL2 integration.
    - Start Docker Desktop and make sure WSL2 integration for your Ubuntu distro is ON.
2. **Open Ubuntu (WSL2) and install tools**

# update packages

sudo apt update && sudo apt upgrade -y

# install basic tools

sudo apt install -y curl wget git jq docker.io

# install Go (needed to build Go chaincode) - example: go1.20

wget https://go.dev/dl/go1.20.8.linux-amd64.tar.gz

sudo tar -C /usr/local -xzf go1.20.8.linux-amd64.tar.gz

export PATH=$PATH:/usr/local/go/bin:$HOME/go/bin

# add to ~/.profile to persist if desired

(If you prefer Node chaincode, install Node and npm instead.)

3. **Clone Fabric samples and download binaries & Docker images**

# clone fabric-samples (this will include test-network)

git clone https://github.com/hyperledger/fabric-samples.git

cd fabric-samples

# Download Fabric binaries & docker images (this script pulls a release)

# Note: the bootstrap script historically lives in fabric-samples. Use the provided script:

curl -sSL https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/bootstrap.sh -o bootstrap.sh

chmod +x bootstrap.sh

# Run bootstrap to get Fabric binaries and docker images (choose a release tag if needed)

./bootstrap.sh 2.4.9 1.5.4 2.4.9

# If you run into version issues, check Fabric docs or the bootstrap.sh usage.

4. **Prepare chaincode folder**

# stay inside fabric-samples

cd test-network

# create a short chaincode project (we'll place a Go chaincode here)

mkdir -p ../chaincode/simpleasset/go

5. **Add the chaincode source**

- Copy the chaincode file (see **Whole code** section) into:
  fabric-samples/chaincode/simpleasset/go/simpleasset.go

6. **Start the test network and create a channel**

# from fabric-samples/test-network

./network.sh up createChannel -c mychannel

This script starts the Fabric docker containers (orderers, peers, CAs) and creates channel mychannel.

7. **Deploy (install & commit) the chaincode**

# from fabric-samples/test-network

# The deployCC helper automates packaging, installing, approving, and committing the chaincode

./network.sh deployCC -ccn simpleasset -ccp ../chaincode/simpleasset/go -ccl go

8. **Interact with the chaincode (invoke & query)**

# Use the pre-provided CLI script to run queries/invokes as Org1's admin

# Example: Invoke CreateAsset and Query

export PATH=$PATH:$PWD/../bin

# create an asset: key "asset1", owner "Alice", value 100

peer lifecycle chaincode querycommitted --channelID mychannel --name simpleasset

# Use the provided invoke.sh or the sample scripts in test-network to call the contract:

./scripts/invoke.sh mychannel org1.example.com simpleasset "CreateAsset" "asset1" "Alice" "100"

# Then query:

./scripts/query.sh mychannel org1.example.com simpleasset "ReadAsset" "asset1"

(If invoke.sh / query.sh are not present in your test-network, you can run the peer CLI commands found in the scripts folder of test-network.)

9. **Shutdown the network (when done)**

# from fabric-samples/test-network

./network.sh down

**Explanation**

What you're doing (overview)

- Using Fabric's **test-network** (local Docker-based network) to run a small, single-channel Fabric network.
- Writing a **very short Go chaincode** (simpleasset) that implements basic asset operations:
  - CreateAsset(id, owner, value)
  - ReadAsset(id)
  - TransferAsset(id, newOwner)
  - AssetExists(id)
- Packaging and deploying the chaincode with the test-network helper network.sh deployCC which runs the Fabric lifecycle steps (package → install → approve → commit).
- Invoking the chaincode with peer CLI scripts to create and read assets.

Why WSL2 on Windows

- Fabric network scripts and Docker tooling are Linux-friendly. WSL2 gives a Linux environment integrated with Windows and Docker Desktop.

Key files & roles

- simpleasset.go — the chaincode source (Go) implementing the smart contract logic using contractapi from github.com/hyperledger/fabric-contract-api-go.
- test-network/network.sh — comes with fabric-samples and automates bringing up the network and deploying chaincode.
- peer CLI (invoked by scripts) — used to interact with the deployed contract.

Important practical notes

- Use the Fabric versions compatible with your bootstrap.sh call. If commands fail, check Fabric docs and match versions of binaries and docker images.
- The deployCC helper in test-network assumes chaincode is under fabric-samples/chaincode/... and that you pass the correct language flag (-ccl go).
- For production or multi-org deployment, you'll need certificates, MSP configuration, anchors, gossip configuration, endorsement policies, and a more complex lifecycle process.

Security & development tips

- This example is for development/testing only.
- Add input validation and access control checks in chaincode for production.
- Use module-aware Go (a go.mod file) when building chaincode for newer Fabric versions — the sample code below is module-compatible.

---

**Whole code**

Save this file as:

fabric-samples/chaincode/simpleasset/go/simpleasset.go

```
package main


import (
	"encoding/json"
	"fmt"


	"github.com/hyperledger/fabric-contract-api-go/contractapi"
)


// SimpleAssetContract provides functions for managing an asset
type SimpleAssetContract struct {
	contractapi.Contract
}


// Asset describes basic details of a simple asset
type Asset struct {
	ID    string `json:"id"`
	Owner string `json:"owner"`
	Value int    `json:"value"`
}


// CreateAsset creates a new asset on the ledger
func (s *SimpleAssetContract) CreateAsset(ctx contractapi.TransactionContextInterface, id string, owner string, value int) error {
	exists, err := s.AssetExists(ctx, id)
	if err != nil {
		return err
	}
	if exists {
		return fmt.Errorf("asset %s already exists", id)
```

```
	}
	asset := Asset{
		ID:    id,
		Owner: owner,
		Value: value,
	}
	assetJSON, err := json.Marshal(asset)
	if err != nil {
		return err
	}
	return ctx.GetStub().PutState(id, assetJSON)
}

// ReadAsset returns the asset stored in the world state with given id
func (s *SimpleAssetContract) ReadAsset(ctx contractapi.TransactionContextInterface, id string) (*Asset, error) {
	assetJSON, err := ctx.GetStub().GetState(id)
	if err != nil {
		return nil, fmt.Errorf("failed to read asset %s from world state: %v", id, err)
	}
	if assetJSON == nil {
		return nil, fmt.Errorf("asset %s does not exist", id)
	}
	var asset Asset
	if err := json.Unmarshal(assetJSON, &asset); err != nil {
		return nil, err
	}
	return &asset, nil
}

// TransferAsset changes the owner of an asset
func (s *SimpleAssetContract) TransferAsset(ctx contractapi.TransactionContextInterface, id string, newOwner string) error {
```

```
	asset, err := s.ReadAsset(ctx, id)
	if err != nil {
		return err
	}
	asset.Owner = newOwner
	assetJSON, err := json.Marshal(asset)
	if err != nil {
		return err
	}
	return ctx.GetStub().PutState(id, assetJSON)
}

// AssetExists returns true when asset with given ID exists in world state
func (s *SimpleAssetContract) AssetExists(ctx contractapi.TransactionContextInterface, id string) (bool, error) {
	assetJSON, err := ctx.GetStub().GetState(id)
	if err != nil {
		return false, fmt.Errorf("failed to read asset %s from world state: %v", id, err)
	}
	return assetJSON != nil, nil
}

func main() {
	chaincode, err := contractapi.NewChaincode(new(SimpleAssetContract))
	if err != nil {
		fmt.Printf("Error creating simpleasset chaincode: %v\n", err)
		return
	}

	if err := chaincode.Start(); err != nil {
		fmt.Printf("Error starting simpleasset chaincode: %v\n", err)
	}
}
```

## 7.Write a smart contract using a solidity program to perform the balance transfer from contract to other accounts

**Steps to execute**

Run these commands in **PowerShell**. Each line is a separate command unless grouped.

1. Install Node.js (if not already)
    - Download & install Node 16+ from https://nodejs.org (enable “Add to PATH”).
    - Verify:
2. node --version
3. npm --version
4. Create project folder and initialize

mkdir C:\transfer-contract

cd C:\transfer-contract

npm init -y

3. Install Hardhat and ethers

npm install --save-dev hardhat @nomiclabs/hardhat-ethers ethers

npx hardhat  # choose "Create a basic sample project" or "Create an empty hardhat.config.js"

4. Create files (see **Whole code** below)

- contracts/ContractTransfer.sol — Solidity contract (short)
- scripts/deploy-and-transfer.js — deploy + fund + transfer demo
- Ensure hardhat.config.js exists (npx hardhat created one). If not, use the default Hardhat config.

5. Run the demo (uses Hardhat's ephemeral local network)

npx hardhat run scripts/deploy-and-transfer.js

You will see console logs showing balances before/after the contract transfer.

(Optional) To run against a persistent local node:

# in terminal 1:

npx hardhat node

# in terminal 2 (deploy to local node):

npx hardhat run scripts/deploy-and-transfer.js --network localhost

---

**Explanation**

What this does

- The Solidity contract ContractTransfer can **receive Ether** and **send Ether** from the contract to other accounts.
- The contract uses an owner (set at deployment) and an onlyOwner modifier so **only the owner** can call transferOut (prevents arbitrary draining).
- receive() fallback lets the contract accept plain ETH transfers.
- transferOut(address payable to, uint256 amount) uses the recommended call pattern to send Ether and checks the success boolean to avoid silent failures and reduce reentrancy risk (no external state changes after the call).
- The Hardhat script:
    - deploys the contract,
    - funds it (owner -> contract),
    - calls transferOut to send ETH from contract to a recipient,
    - prints balances (owner, contract, recipient) before and after.

Security notes (brief)

- Use onlyOwner to restrict who can withdraw. In production, consider multisig or time locks.
- Use call{value: amount}("") and check return value (done here).
- Avoid making state changes after external calls (this contract does not make any after the call).
- For full safety, add reentrancy guard and input validation in more complex contracts.

Why this is Windows-only

- Steps show PowerShell commands and Hardhat usage on Windows. Hardhat works cross-platform.

---

**Whole code (two files)**

Save these files under C:\transfer-contract.

**contracts/ContractTransfer.sol**

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;


/// @title Contract that holds ETH and allows owner to transfer out

contract ContractTransfer {

    address public owner;


    event Received(address indexed sender, uint amount);

    event Transferred(address indexed to, uint amount);
```

```
    modifier onlyOwner() {
        require(msg.sender == owner, "only owner");
        _;
    }

    constructor() {
        owner = msg.sender;
    }

    // Allow contract to receive ETH
    receive() external payable {
        emit Received(msg.sender, msg.value);
    }

    // Owner can transfer ETH from contract to another address
    function transferOut(address payable to, uint256 amount) external onlyOwner {
        require(address(this).balance >= amount, "insufficient contract balance");
        (bool success, ) = to.call{value: amount}("");
        require(success, "transfer failed");
        emit Transferred(to, amount);
    }

    // convenience: check contract balance
    function contractBalance() external view returns (uint256) {
        return address(this).balance;
    }
}
```

---

**scripts/deploy-and-transfer.js**

```
// scripts/deploy-and-transfer.js
const hre = require("hardhat");
```

```
async function main() {
  const [owner, recipient] = await hre.ethers.getSigners();

  // Deploy contract
  const Factory = await hre.ethers.getContractFactory("ContractTransfer");
  const contract = await Factory.deploy();
  await contract.deployed();
  console.log("Contract deployed at:", contract.address);
  console.log("Owner address:", owner.address);
  console.log("Recipient address:", recipient.address);

  const toEther = (wei) => hre.ethers.utils.formatEther(wei);

  // helper: print balances
  async function printBalances(stage) {
    const ownerBal = await owner.getBalance();
    const contractBal = await hre.ethers.provider.getBalance(contract.address);
    const recipBal = await recipient.getBalance();
    console.log(`\n== ${stage} ==`);
    console.log("Owner:", toEther(ownerBal));
    console.log("Contract:", toEther(contractBal));
    console.log("Recipient:", toEther(recipBal));
  }

  await printBalances("initial");

  // Fund contract from owner: send 1.0 ETH
  const fundTx = await owner.sendTransaction({
    to: contract.address,
    value: hre.ethers.utils.parseEther("1.0")
  });
  await fundTx.wait();
```

```
  await printBalances("after funding contract");

  // Owner instructs contract to transfer 0.5 ETH to recipient
  const transferAmount = hre.ethers.utils.parseEther("0.5");
  const tx = await contract.connect(owner).transferOut(recipient.address, transferAmount);
  await tx.wait();
  await printBalances("after transferOut");

  // Done
  console.log("\nDemo complete.");
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

## 8. Write a program to perform token Creation and Management on Ethereum.

## Steps to execute

Run these commands in **PowerShell** (each line is a separate command unless grouped).

1. Install Node.js (if not already)
    - Download & install Node 16+ from https://nodejs.org (enable “Add to PATH”).
    - Verify:
2. node --version
3. npm --version
4. Create project folder and initialize

mkdir C:\token-demo

cd C:\token-demo

npm init -y

3. Install Hardhat, ethers and OpenZeppelin contracts

npm install --save-dev hardhat @nomiclabs/hardhat-ethers ethers

npm install @openzeppelin/contracts

npx hardhat # choose "Create a basic sample project" or "Create an empty hardhat.config.js"

4. Create files (see **Whole code** below):

- contracts/MyToken.sol — ERC20 token using OpenZeppelin (very short)
- scripts/deploy-and-manage.js — deploy + mint + transfer + approve demo

5. Run the demo on Hardhat’s local network:

npx hardhat run scripts/deploy-and-manage.js

(That runs an ephemeral in-memory node, deploys the token, and prints balances/allowances.)

(Optional) To run against a persistent local node:

# Terminal 1

npx hardhat node

# Terminal 2

npx hardhat run scripts/deploy-and-manage.js --network localhost

---

**Explanation**

What this does (overview)

- We create a standard ERC-20 token by inheriting OpenZeppelin’s ERC20 and Ownable.

- The contract exposes:
    - mint(address to, uint256 amount) — only owner can mint (create) tokens.
    - burn(uint256 amount) — token holder can burn (destroy) their tokens.
- The demo script:
    - deploys the token,
    - mints tokens to Owner and User1,
    - transfers tokens from Owner → User2,
    - approves a spender and checks allowance,
    - has spender transferFrom using allowance,
    - prints balances and allowance at each step.

Why use OpenZeppelin

- Saves time and reduces risk: OZ's audited ERC-20 implementation follows the standard and handles edge cases (events, decimals, etc.).

Key security notes (brief)

- mint is onlyOwner in this sample — in real projects consider role-based minting, timelocks, or governance.
- Do not use the same private keys or RPC endpoints on mainnet without auditing and proper deployment practices.
- This sample runs locally (Hardhat). To deploy to testnet/mainnet you must configure RPC URL and private key in Hardhat config and .env (not shown here for brevity).

---

**Whole code (two files)**

Create these files under C:\token-demo.

**contracts/MyToken.sol**

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;


/// Very short ERC20 token using OpenZeppelin

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

import "@openzeppelin/contracts/access/Ownable.sol";


contract MyToken is ERC20, Ownable {

   constructor(string memory name_, string memory symbol_) ERC20(name_, symbol_) {

      // optionally mint initial supply in constructor if desired
```

```
    }

    // owner-only minting
    function mint(address to, uint256 amount) external onlyOwner {
        _mint(to, amount);
    }

    // allow holders to burn their own tokens
    function burn(uint256 amount) external {
        _burn(msg.sender, amount);
    }
}
```

---

**scripts/deploy-and-manage.js**

```
// scripts/deploy-and-manage.js
const hre = require("hardhat");

async function main() {
  const [owner, user1, user2, spender] = await hre.ethers.getSigners();

  const Token = await hre.ethers.getContractFactory("MyToken");
  const token = await Token.deploy("DemoToken", "DMT");
  await token.deployed();
  console.log("Token deployed at:", token.address);

  const toEther = (raw) => hre.ethers.utils.formatUnits(raw, 18);

  // helper to print balances
  async function printBalances(stage) {
    const bOwner = await token.balanceOf(owner.address);
    const b1 = await token.balanceOf(user1.address);
    const b2 = await token.balanceOf(user2.address);
```

```
  const allowance = await token.allowance(user1.address, spender.address);
  console.log(`\n== ${stage} ==`);
  console.log("Owner:", toEther(bOwner));
  console.log("User1:", toEther(b1));
  console.log("User2:", toEther(b2));
  console.log("Allowance(User1->Spender):", toEther(allowance));
 }

 // mint tokens: remember ERC20 has 18 decimals by default
 const mintAmount = hre.ethers.utils.parseUnits("1000", 18);
 await token.connect(owner).mint(owner.address, mintAmount);
 await token.connect(owner).mint(user1.address, hre.ethers.utils.parseUnits("100", 18));
 await printBalances("after mint");

 // transfer tokens from owner to user2
 await token.connect(owner).transfer(user2.address, hre.ethers.utils.parseUnits("50", 18));
 await printBalances("after owner -> user2 transfer (50)");

 // user1 approves spender for 30 tokens
 await token.connect(user1).approve(spender.address, hre.ethers.utils.parseUnits("30", 18));
 await printBalances("after user1 approves spender (30)");

 // spender uses transferFrom to move 20 tokens from user1 -> user2
 await token.connect(spender).transferFrom(user1.address, user2.address,
hre.ethers.utils.parseUnits("20", 18));
 await printBalances("after spender transferFrom 20");

 // user1 burns 10 tokens
 await token.connect(user1).burn(hre.ethers.utils.parseUnits("10", 18));
 await printBalances("after user1 burns 10");

 console.log("\nDemo complete.");
}
```

```
main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

## 9. Setup Metamask in the System and Create a wallet in the Metamask with Test Network

**Steps to execute**

Follow these in **PowerShell** + your browser. Each step is independent.

1. **Install a supported browser**
    - Use Chrome, Edge, or Firefox for best compatibility. (If you already have one, skip.)
2. **Install MetaMask extension**
    - Open your browser, go to MetaMask download page and pick your browser: https://metamask.io/download. MetaMask
    - Click the store link (Chrome Web Store / Edge Add-ons / Firefox Add-ons) and **Add extension**. Confirm the install.
3. **Create a new MetaMask wallet**
    - Click the MetaMask icon (top-right of browser).
    - Click **Get Started → Create a Wallet**.
    - Choose whether to help improve MetaMask (optional).
    - Create a strong password for this browser extension (used only locally).
    - MetaMask will show your **Secret Recovery Phrase (12 words) — WRITE IT DOWN on paper** and store it securely offline. Do **not** store it in plain text or share it. MetaMask will ask you to confirm the phrase by selecting words in order. Finish setup.
4. **Enable and switch to a test network (Sepolia)**
    - Open MetaMask, click the network dropdown (it may show “Ethereum Mainnet”).
    - If you don’t see testnets like **Sepolia**, enable them:
        - Click your account icon → **Settings → Advanced** → toggle **Show test networks** on.
    - Now select **Sepolia** from the network dropdown.
5. **Get Sepolia test ETH (faucet)**
    - Use an official faucet. Options include Alchemy’s Sepolia faucet or Chainlink’s Sepolia faucet. You’ll need to paste your MetaMask wallet address and request ETH. Examples:
        - Alchemy Sepolia Faucet (requires Alchemy account): Alchemy faucets page. Alchemy+1
        - Chainlink Sepolia Faucet: https://faucets.chain.link/sepolia. Chainlink Faucets
    - After requesting, the faucet will send test ETH to your Sepolia address. Check your MetaMask balance (Sepolia network).
6. **(Optional) Importing an existing wallet**
    - If you already have a seed phrase, in MetaMask choose **Import wallet** and paste the Secret Recovery Phrase (seed). Create a new extension password and finish.
7. **Quick test: connect a webpage to MetaMask**

- Save the sample HTML (see **Whole code** below) as connect.html.
- Open it in your browser and click **Connect Wallet** — MetaMask will prompt to connect; approve and the page will display your address.

---

**Explanation**

What we did and why

- **MetaMask** is a browser extension wallet that holds private keys locally and lets you interact with dapps (web3 sites). Install from the official site to avoid fake extensions.
- **Secret Recovery Phrase** (seed) is the master key for your wallet. Anyone with it can access your funds — store it offline and never share. MetaMask never stores this server-side.
- **Test networks** like **Sepolia** let you experiment without real ETH. Faucets give small amounts of test ETH for development/testing. Use faucets such as Alchemy's Sepolia faucet or Chainlink's faucet.
- The **webpage program** uses the standard window.ethereum (MetaMask) provider to request account access (eth_requestAccounts) and display the connected account. This is the simplest pattern dapps use to get the user's address and interact with the wallet.

Security reminders (must-read)

- **Never** share your Secret Recovery Phrase or private keys. Anyone with them can steal your assets.
- Use test networks for development; do not put real funds in test workflows.
- When connecting to unknown dapps, inspect permissions — a dapp can request account access and ask you to sign transactions; signing arbitrary messages/transactions can be risky.

---

**Whole code**

Below are two tiny pieces of code:

A) A **PowerShell** helper that opens the MetaMask download page and your browser extensions page (very short, Windows-friendly). Save as open-metamask.ps1 and run from PowerShell.

B) A minimal **HTML + JavaScript** page (connect.html) that connects to MetaMask and shows your account address. Save and open in your browser (after MetaMask is installed).

---

**A) PowerShell helper — open-metamask.ps1**

```
# open-metamask.ps1

# Very short helper: opens MetaMask download page and browser extensions/settings

# Usage: Right-click -> Run with PowerShell, or run in PowerShell: .\open-metamask.ps1


# Open MetaMask official download page

Start-Process "https://metamask.io/download"
```

```
# Optionally open Chrome extensions page (uncomment if you use Chrome)
# Start-Process "chrome://extensions/"

# Optionally open Edge extensions page (uncomment if you use Edge)
# Start-Process "edge://extensions/"

Write-Host "Opened MetaMask download page in your default browser."
Write-Host "If you already have MetaMask installed, open the extension (top-right) to finish setup."
```

Note: Start-Process "chrome://extensions/" works if Chrome is default; otherwise open manually. PowerShell cannot silently install a browser extension — extension install must be approved by the user.

---

**B) Minimal HTML page to connect to MetaMask — connect.html**

Save this file and open it in your browser (with MetaMask installed). It prompts MetaMask to connect and shows your first account address.

```
<!doctype html>
<html>
 <head>
  <meta charset="utf-8" />
  <title>Connect MetaMask (very short)</title>
  <style>
   body { font-family: Arial, sans-serif; padding: 20px; }
   button { padding: 8px 12px; font-size: 16px; }
   pre { background:#f4f4f4; padding:10px; border-radius:6px; }
  </style>
 </head>
 <body>
  <h2>Connect MetaMask (Sepolia / Testnet)</h2>
  <button id="connect">Connect Wallet</button>
  <div id="result"></div>

  <script>
   const btn = document.getElementById('connect');
```

```
    const result = document.getElementById('result');


    btn.onclick = async () => {
      if (typeof window.ethereum === 'undefined') {
        result.innerHTML = '<p style="color:darkred">MetaMask not detected. Install it from <a href="https://metamask.io/download" target="_blank">metamask.io</a>.</p>';
        return;
      }
      try {
        // request account access
        const accounts = await window.ethereum.request({ method: 'eth_requestAccounts' });
        const account = accounts[0];
        result.innerHTML = `<p>Connected account:</p><pre>${account}</pre>`;
        // show network chain id (optional)
        const chainId = await window.ethereum.request({ method: 'eth_chainId' });
        result.innerHTML += `<p>Chain ID: ${chainId}</p>`;
      } catch (err) {
        result.innerHTML = `<p style="color:darkred">Connection rejected or error: ${err.message}</p>`;
      }
    };
  </script>
 </body>
</html>
```

How to use the HTML file

1. Save as connect.html.
2. Open your browser (with MetaMask installed and unlocked).
3. File → Open File (or drag it into a tab).
4. Click **Connect Wallet**. MetaMask will prompt you to allow the site to connect — approve to see your account address displayed.

---

## 10. Develop a program to implement blockchain in Merkle Trees.

**Steps to execute**

1. **Install Python (if you don't have it)**
    - Download & install Python 3.8+ from https://python.org and check **"Add Python to PATH"** during install.
    - Verify in PowerShell:
    - python --version
2. **Create a folder for the project**
3. mkdir C:\merkle-blockchain
4. cd C:\merkle-blockchain
5. **Save the script**
    - Create a file named merkle_blockchain.py in C:\merkle-blockchain.
    - Copy the code from the **Whole code** section below into that file and save it.
6. **Run the program**
7. cd C:\merkle-blockchain
8. python merkle_blockchain.py
    - The script prints the chain, each block's merkle root, and verifies the merkle root for each block.

---

**Explanation (short & separate)**

What this program demonstrates

- A tiny blockchain where each block stores:
    - transactions (a list of strings),
    - a **Merkle root** computed from those transactions,
    - previous_hash, timestamp, and the block's own hash.
- The **Merkle tree** is used to summarize many transactions into a single root hash: leaves are transaction hashes; internal nodes are hash(left + right). If a level has an odd number of nodes, the last node is duplicated before hashing (a common simple approach).
- Merkle roots allow efficient proofs of inclusion: to prove a transaction is part of a block you only need the branch ($\log_2(n)$ hashes) rather than the whole list.
- The program:
    - builds Merkle root for each block,
    - stores it in the block,

- computes the block hash from (index, timestamp, merkle_root, previous_hash) (so if transactions change the merkle root changes and block hash changes),
- verifies for every block that the stored merkle root equals a fresh recomputation (integrity check).

Why Merkle trees matter in blockchains

- Compactly represent many transactions.
- Enable light clients (SPV) to verify inclusion without downloading entire blocks.
- Make tamper-detection efficient: changing or removing any transaction changes the merkle root and thus the block hash.

Limitations & notes

- This is an educational, single-node prototype:
  - No transaction signatures, no networking, no consensus (single-authority chain).
  - Merkle proof generation/verification for a single transaction is not included but is straightforward to add.
- For production you'd use a canonical hashing scheme and consistent duplication/padding rules (and typically more robust merkle libraries).

---

**Whole code (save as merkle_blockchain.py)**

```
# merkle_blockchain.py
# Very short blockchain demo using Merkle Trees (Windows-friendly)
# Run: python merkle_blockchain.py

import hashlib, json, time
from typing import List

def sha256(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def hash_pair(a: str, b: str) -> str:
    """Hash two hex strings together."""
    return sha256((a + b).encode())

def merkle_root(txns: List[str]) -> str:
    """
```

```
    Compute a simple Merkle root from a list of transaction strings.
    If a level has an odd number of nodes, duplicate the last one.
    """
    if not txns:
        return sha256(b'')  # empty root
    # leaves: hash each transaction
    level = [sha256(tx.encode()) for tx in txns]
    while len(level) > 1:
        next_level = []
        for i in range(0, len(level), 2):
            left = level[i]
            right = level[i+1] if i+1 < len(level) else level[i]  # duplicate if odd
            next_level.append(hash_pair(left, right))
        level = next_level
    return level[0]


class Block:
    def __init__(self, index: int, transactions: List[str], previous_hash: str):
        self.index = index
        self.timestamp = time.time()
        self.transactions = transactions
        self.merkle_root = merkle_root(transactions)
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self) -> str:
        payload = {
            'index': self.index,
            'timestamp': self.timestamp,
            'merkle_root': self.merkle_root,
            'previous_hash': self.previous_hash
        }
```

```
        return sha256(json.dumps(payload, sort_keys=True).encode())

    def verify_merkle(self) -> bool:
        """Recompute merkle root and compare to stored value."""
        return merkle_root(self.transactions) == self.merkle_root

class SimpleMerkleBlockchain:
    def __init__(self):
        self.chain: List[Block] = [self.create_genesis()]

    def create_genesis(self) -> Block:
        return Block(0, ["genesis"], "0")

    def add_block(self, transactions: List[str]):
        prev = self.chain[-1]
        block = Block(len(self.chain), transactions, prev.hash)
        self.chain.append(block)
        return block

    def is_chain_valid(self) -> bool:
        for i in range(1, len(self.chain)):
            block = self.chain[i]
            prev = self.chain[i-1]
            if block.previous_hash != prev.hash:
                print(f"Block {i}: previous_hash mismatch")
                return False
            if block.hash != block.calculate_hash():
                print(f"Block {i}: hash recalculation mismatch")
                return False
            if not block.verify_merkle():
                print(f"Block {i}: merkle root invalid")
                return False
```

```
        return True


# Demo
def demo():
    bc = SimpleMerkleBlockchain()

    # Add blocks with varying transaction counts
    bc.add_block(["Alice->Bob:5", "Charlie->Dan:2", "Eve->Frank:10"])
    bc.add_block(["Tx1", "Tx2", "Tx3", "Tx4", "Tx5"])  # odd number -> duplication happens
    bc.add_block([])  # empty transactions allowed (demonstrates empty root)

    # Print summary
    for b in bc.chain:
        print(f"Block {b.index}:")
        print(f"  timestamp: {time.ctime(b.timestamp)}")
        print(f"  previous_hash: {b.previous_hash[:12]}...")
        print(f"  merkle_root: {b.merkle_root}")
        print(f"  tx_count: {len(b.transactions)}")
        print(f"  hash: {b.hash}\n")
     print("Chain valid?:", bc.is_chain_valid())

    # Demonstrate tamper detect: change a transaction then re-check
    print("\n-- Tamper test --")
    if len(bc.chain) > 1 and bc.chain[1].transactions:
        print("Original tx[0]:", bc.chain[1].transactions[0])
        bc.chain[1].transactions[0] = "Alice->Bob:5000"  # tamper
        print("Tampered tx[0]:", bc.chain[1].transactions[0])
        print("Merkle valid after tamper?:", bc.chain[1].verify_merkle())
        print("Chain valid after tamper?:", bc.is_chain_valid())

if __name__ == "__main__":
    demo()
```